

# One-to-one SkewGRAM : from biology to NLP

Bechir Mady<sup>1</sup>, Abderahmane Abderrahmane<sup>1</sup>, El Moustapha Mohamed El Moustapha<sup>1</sup>, and Mohamedou Yahya Cheikh Mohamed Vall<sup>1</sup>

Institut Supérieur du Numérique (SupNum), Nouakchott, Mauritanie  
{22014, 22053, 22013, 24239}@supnum.mr

**Résumé** Le problème One-to-one SkewGRAM est un défi d’optimisation combinatoire NP-difficile issu de la bio-informatique. Il vise à trouver le plus long chemin dans un graphe orienté acyclique  $D$  tel que ses sommets induisent un sous-graphe connexe dans un graphe non-orienté  $G$ . Les méthodes exactes (ILP) sont exponentiellement lentes, et les heuristiques basées sur les Réseaux de Neurones sur Graphes (GNN) génèrent des chemins trivialement courts ( $\approx 1.2$  nuds). Nous proposons **One-to-one Skew-GRAM**, un modèle d’embeddings de nuds de type Skip-Gram (paradigme DeepWalk/node2vec) entraîné sur des marches aléatoires dans  $D$  avec échantillonnage négatif structurel, couplé à un décodeur guidé combinant à chaque pas trois signaux : (i) une anticipation exacte par programmation dynamique du plus long chemin restant dans le DAG  $D$ , qui évite à la marche de s’enliser dans des culs-de-sac, (ii) la similarité d’embeddings apprise, et (iii) un bonus récompensant les successeurs qui préservent la connexité dans  $G$ . Une étape finale exacte extrait, via un Union-Find à fenêtre glissante, le plus long sous-chemin contigu strictement connexe dans  $G$ . Évaluée sur 38 instances synthétiques (100 nuds, dont 36 avec un optimum ILP2 connu), notre approche résout chaque instance en **2.7 secondes** en moyenne (soit  $\approx 10\times$  plus rapide que le solveur exact ILP2), tout en découvrant des chemins de **7.13 nuds** de moyenne (écart moyen de  $-30\%$  à l’optimum ILP2, et égalité ou dépassement sur 13/36 instances), surpassant de très loin l’état de l’art GNN et offrant un bon compromis qualité-temps pour le passage à l’échelle génomique.

**Keywords** : Optimisation Combinatoire · Bio-informatique · Algorithmes sur Graphes · Embeddings de Nuds · Skip-Gram · Programmation Dynamique · Composantes Connexes · Union-Find.

## 1 Introduction

L’intelligence artificielle (IA) et l’apprentissage automatique connaissent une adoption croissante dans le domaine de la bio-informatique. Dans les pays en développement, l’adoption d’approches basées sur l’IA offre un potentiel unique pour contourner les stades traditionnels de développement et accélérer les découvertes biologiques, notamment dans le traitement des maladies ou l’agriculture. Un défi majeur dans ce contexte est l’intégration et l’analyse de multiples couches de données biologiques (multi-omiques).

Les cellules vivantes sont régies par de multiples réseaux d’interactions. Par exemple, le métabolisme cellulaire peut être modélisé comme un graphe orienté, où les métabolites sont reliés par des réactions biochimiques dirigées, illustrant un flux causal. Simultanément, la proximité génomique ou les interactions protéine-protéine (PPI) peuvent être modélisées par des graphes non-orientés, décrivant des associations fonctionnelles fortes. L’identification de sous-structures communes à ces deux graphes permet de découvrir des modules fonctionnels essentiels conservés au cours de l’évolution.

Le problème **One-to-one SkewGRAM** formalise ce défi : trouver un chemin orienté dans le graphe métabolique dont les entités forment un tout connexe dans le réseau fonctionnel. Ce problème est NP-difficile.

Les approches récentes ont tenté d’utiliser les Réseaux de Neurones sur Graphes (GNN) [1]. Bien que prometteuses, ces méthodes génèrent des chemins très courts (1.2 nuds en moyenne), car l’espace d’exploration combinatoire est trop vaste pour un décodage séquentiel non contraint. Le solveur exact ILP2 [1] est optimal mais très coûteux en temps (28 secondes par instance de 100 nuds).

Dans cet article, nous proposons **One-to-one Skew-GRAM**, une approche qui transpose le principe du Skip-Gram (Mikolov et al., 2013), via le paradigme DeepWalk/node2vec (Perozzi et al., 2014; Grover & Leskovec, 2016), à ce problème d’optimisation combinatoire sur deux graphes couplés. Les embeddings de nuds sont entraînés uniquement sur des marches dans  $D$ , puisque le chemin cherché doit lui-même être un chemin valide dans  $D$ ; la contrainte imposée par  $G$  n’est injectée qu’au moment du décodage, à la fois comme signal de guidage souple pendant la génération et comme contrainte dure, vérifiée exactement, dans une étape finale d’extraction.

Les contributions de cet article sont :

1. Un modèle d’embeddings de type Skip-Gram pour les nuds d’un DAG, utilisant un échantillonnage négatif structurel (non-successeurs dans  $D$ ) et un sous-échantillonnage des hubs.
2. Un décodeur guidé combinant une anticipation exacte du plus long chemin dans le DAG, la similarité d’embeddings apprise et un bonus de connexité à  $G$ , suivi d’une extraction exacte par Union-Find à fenêtre glissante du plus long sous-chemin  $(D,G)$ -consistant.
3. Une validation empirique sur 38 instances démontrant une accélération de  $\sim 10\times$  par rapport à ILP2, avec des chemins moyens de 7.13 nuds (contre 9.97 pour l’optimum ILP2), égalant ou dépassant ILP2 sur 13/36 instances, et surpassant de très loin la référence GNN à 1.2 nud.

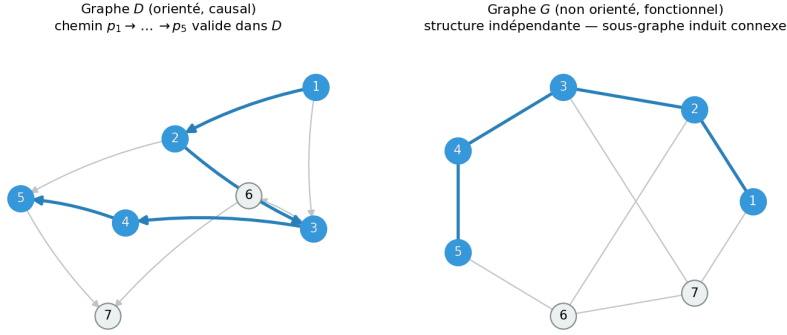
## 2 Formulation du Problème et Propriété Mathématique

Formellement, soit  $V = \{v_1, \dots, v_n\}$  un ensemble de nuds (entités biologiques).

- $D = (V, A)$  : graphe orienté acyclique (DAG) causal.
- $G = (V, E)$  : graphe non-orienté fonctionnel.

**Objectif :** Trouver un chemin  $P = (p_1, \dots, p_k)$  maximisant  $k$ , sous les conditions : 1.  $\forall i, (p_i, p_{i+1}) \in A$  (valide dans  $D$ ). 2. Le sous-graphe  $G[\{p_1, \dots, p_k\}]$  est connexe. Un tel chemin est **(D,G)-consistant**.

$D$  et  $G$  partagent les mêmes nœuds mais pas la même structure



**Figure 1.** Exemple jouet sur 7 nuds.  $D$  et  $G$  partagent le même ensemble de nuds mais ont des topologies par ailleurs indépendantes (arêtes différentes, forme différente). Le même chemin  $(p_1, \dots, p_5)$  doit à la fois être un chemin orienté valide dans  $D$  (gauche) et induire un sous-graphe connexe dans  $G$  (droite), bien que  $D$  et  $G$  n'aient visuellement rien en commun.

## 2.1 La Connexité ne Peut pas Être Vérifiée Incrémentalement

Un audit empirique des solutions optimales ILP2 montre que le préfixe de nuds visités à une étape intermédiaire n'induit presque jamais, à lui seul, un sous-graphe connexe de  $G$  : la connexité est une propriété de l'*ensemble final* de nuds, et non une propriété qui peut être garantie pas à pas pendant la construction du chemin. Par conséquent, un décodeur ne peut pas simplement filtrer les successeurs candidats par un test de connexité incrémental ; il doit plutôt générer un chemin candidat dans  $D$  puis vérifier, *a posteriori*, quelle portion contiguë de ce chemin est (D,G)-consistante. Ce constat motive directement le décodeur en deux étapes de la Section 3.

## 3 Méthodologie : One-to-one Skew-GRAM

Notre approche comporte deux composantes : (1) un modèle d'embeddings entraîné une fois par instance sur des marches aléatoires dans  $D$ , et (2) un décodeur guidé qui exploite ces embeddings, ainsi qu'une anticipation exacte sur

le DAG et un signal de connexité à  $G$ , pour générer puis valider des chemins  $(D, G)$ -consistants (Figure 3).

### 3.1 Démarche générale : de l'instance au chemin, via Encoder et Decoder

À haut niveau, la démarche suit exactement le schéma **Encoder**  $\rightarrow$  **Instance Skip-Gram**  $\rightarrow$  **Decoder** : un sommet est un mot, une marche dans  $D$  est une phrase, et le contexte d'un mot vient à la fois de  $D$  (sa position dans les marches) et de  $G$  (ses connexions fonctionnelles).

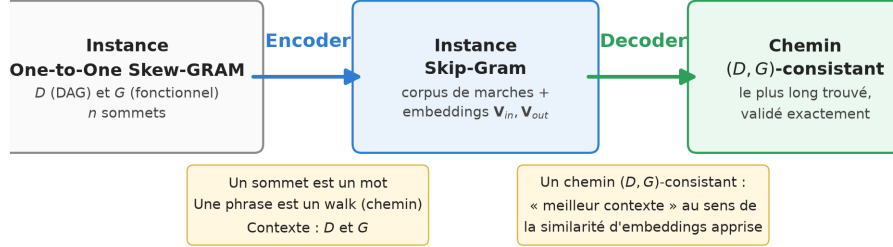
**Encoder.** L'instance  $(D, G)$  est d'abord traduite en tâche de langage. On génère des marches aléatoires dans  $D$  (Phase 1) : depuis un sommet, on avance au hasard parmi ses successeurs, en ne suivant que les arcs sortants – puisque le chemin final doit être valide dans  $D$ , l'espace exploré à ce stade est structuré par  $D$ , pas encore par  $G$ . Les sommets de haut degré sortant, sur-représentés dans les marches comme le seraient des mots vides dans un texte, sont sous-échantillonnés par la formule word2vec standard. Sur chaque marche restante, une fenêtre glissante de taille 3 produit les paires (centre, contexte) qui constituent, concrètement, l'instance Skip-Gram en sortie de l'Encoder.

**Entraînement.** Deux matrices d'embeddings  $\mathbf{V}_{in}, \mathbf{V}_{out}$  (dimension 32) sont entraînées par negative sampling (Phase 2) : chaque paire positive rapproche ses deux vecteurs, chaque négatif les éloigne. La différence avec un Skip-Gram de texte classique est l'échantillonnage négatif *structurel* : les négatifs sont tirés parmi les non-successeurs du centre dans  $D$ , ancrant le signal négatif dans la causalité du graphe plutôt que dans une statistique de fréquence agnostique de la topologie. Ce cycle – marches, corpus, entraînement – est répété séparément pour chaque instance : les identifiants de sommets ne se correspondent pas d'une instance Erds–Rényi à l'autre, donc un embedding appris sur l'une ne transfère pas vers une autre.

**Decoder.** À partir des embeddings appris, on reconstruit un chemin pas à pas en combinant trois signaux à chaque décision (Phase 3) : (i) une anticipation exacte – calculée une fois par instance, en temps linéaire – de la longueur du plus long chemin encore possible dans  $D$  depuis chaque sommet candidat, qui évite les culs-de-sac (sans elle, le décodage guidé par la seule similarité d'embeddings s'arrête en moyenne après 2,9 sommets, contre 12–13 possibles) ; (ii) la similarité d'embeddings elle-même – le sommet suivant est celui dont le vecteur forme le « meilleur contexte » avec le sommet courant ; (iii) un bonus lorsque le candidat est déjà connecté, dans  $G$ , à un sommet du chemin en cours de construction – c'est ici, et seulement ici, que  $G$  intervient dans le décodage. Cette marche guidée est répétée 30 000 fois par instance depuis des départs variés, et le meilleur chemin brut est conservé.

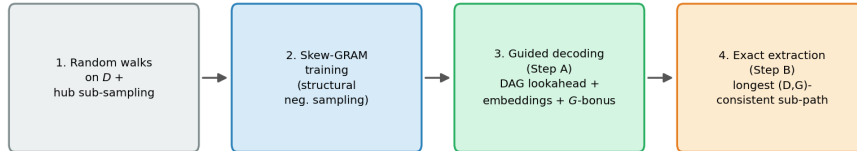
**Validation.** Rien dans les trois signaux ci-dessus ne garantit la  $(D, G)$ -consistance : ils orientent la recherche, ils ne la contraignent pas. Un audit empirique des solutions ILP2 optimales confirme d'ailleurs que la connexité ne peut pas être imposée pas à pas – le préfixe visité à une étape intermédiaire n'induit presque jamais, à lui seul, un sous-graphe connexe de  $G$  (§2.1) ; ce n'est qu'une

propriété de l'ensemble final. La Phase 4 extrait donc, *exactement*, le plus long sous-chemin contigu du chemin brut dont les sommets induisent un sous-graphe connexe dans  $G$ , via un Union-Find à fenêtre glissante – transformant un chemin plausible en chemin garanti (Figure 2).



**Figure 2.** Démarche générale de la solution. L’Encoder (Phases 1–2, §3) traduit l’instance en instance Skip-Gram ; le Decoder (Phases 3–4) la retraduit en chemin  $(D, G)$ -consistant. Valeurs des hyperparamètres en Annexe A.

Les quatre phases détaillées ci-dessous (§3) mettent en uvre concrètement ces deux étapes.



**Figure 3.** Le pipeline One-to-one Skew-GRAM en quatre phases, exécuté une fois par instance.

### 3.2 Phase 1 : Marches Aléatoires et Sous-échantillonnage des Hubs sur $D$

Les marches aléatoires (à la DeepWalk) suivent uniquement les *arcs sortants* de  $D$ , puisque le chemin final doit être valide dans  $D$  ; l’espace de recherche est donc structuré par  $D$ , pas par  $G$ . Les nuds de haut degré sont sous-pondérés via

la formule de sous-échantillonnage de word2vec, réduisant leur domination dans les couples d'entraînement extraits par fenêtre glissante sur les marches.

### 3.3 Phase 2 : Modèle d'Embeddings One-to-one Skew-GRAM

Deux matrices d'embeddings  $\mathbf{V}_{in}, \mathbf{V}_{out} \in \mathbb{R}^{N \times d}$  sont entraînées avec une perte de Negative Sampling :

$$\mathcal{L}(c, o, n_{1:K}) = -\log \sigma(\mathbf{v}_{in,c}^\top \mathbf{v}_{out,o}) - \sum_{k=1}^K \log \sigma(-\mathbf{v}_{in,c}^\top \mathbf{v}_{out,n_k}) \quad (1)$$

La principale différence avec un Skip-Gram classique est l'**échantillonnage négatif structurel** : les négatifs sont tirés parmi les non-successeurs de  $c$  dans  $D$ , plutôt que depuis une distribution unigramme agnostique de la topologie, rendant le signal négatif cohérent avec la structure causale du DAG.

### 3.4 Phase 3 : Décodage Guidé Conscient du DAG (Étape A)

Un décodeur naïf qui échantillonne les successeurs uniquement par similarité d'embeddings s'enlise fréquemment dans des culs-de-sac de  $D$  : sur une instance représentative de 100 nuds, les marches non guidées n'atteignent que 2.9 nuds en moyenne, contre un plus long chemin réel de 12 à 13 nuds. Nous précalculons donc, une fois par instance, une anticipation exacte par programmation dynamique

$$\text{lf}(v) = \max_{w \in N^+(v)} (1 + \text{lf}(w)), \quad \text{lf}(v) = 0 \text{ si } N^+(v) = \emptyset, \quad (2)$$

calculée en  $O(|V| + |A|)$  par une unique passe topologique inversée, donnant la longueur du plus long chemin de  $v$  vers un puits de  $D$ . À chaque pas de décodage depuis le nud  $u$ , un successeur  $v$  est échantillonné parmi  $N^+(u)$  avec une probabilité proportionnelle à  $\exp(\text{lf}(v)/\tau_f + \beta \text{sim}(u, v)/\tau + \log(\gamma) \cdot \mathbb{1}[v \text{ adjacent dans } G \text{ au chemin déjà construit}])$ , combinant l'anticipation sur le DAG (évite les culs-de-sac), la similarité d'embeddings apprise (signal Skew-GRAM) et un bonus  $\gamma$  pour les successeurs qui préservent la connexité dans  $G$ . De nombreuses marches sont échantillonnées depuis des nuds de départ aléatoires, en masquant les nuds déjà visités pour garantir une correspondance one-to-one position nud.

### 3.5 Phase 4 : Extraction Exacte du Sous-chemin (D,G)-Consistant (Étape B)

Comme établi en Section 2.1, la connexité ne peut pas être imposée incrémentalement. Pour chaque chemin candidat  $P$  produit par l'étape A, nous appliquons un algorithme de fenêtre glissante couplé à un Union-Find incrémental pour extraire le sous-intervalle contigu maximal  $[i, j]$  tel que  $G[\{p_i, \dots, p_j\}]$  soit strictement connexe, en  $O(L^2 \alpha(L))$  pour un candidat de longueur  $L$ . Un sous-chemin contigu d'un chemin dans  $D$  est lui-même un chemin valide dans  $D$ , donc cette étape n'impose que la contrainte de connexité dans  $G$  *a posteriori* ; elle n'invalide jamais la validité dans  $D$  établie à l'étape A.

## 4 Résultats et Discussion

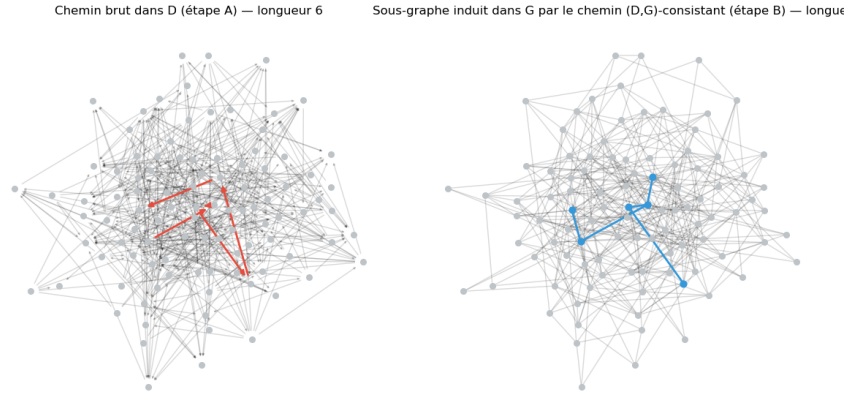
**Table 1.** Comparaison des méthodes sur 38 instances synthétiques (100 nuds, dont 36 avec un optimum ILP2 connu).

| Méthode                                     | Longueur moy.      | Temps moy.   | Accélération                      |
|---------------------------------------------|--------------------|--------------|-----------------------------------|
| GNN DL2 [1] (état de l’art IA)              | $\approx 1.2$ nuds | N/A          |                                   |
| ILP2 [1] (Solveur exact)                    | 9.97 nuds          | 28.0 s       | référence                         |
| <b>One-to-one Skew-GRAM (notre méthode)</b> | <b>7.13 nuds</b>   | <b>2.7 s</b> | <b><math>\sim 10\times</math></b> |

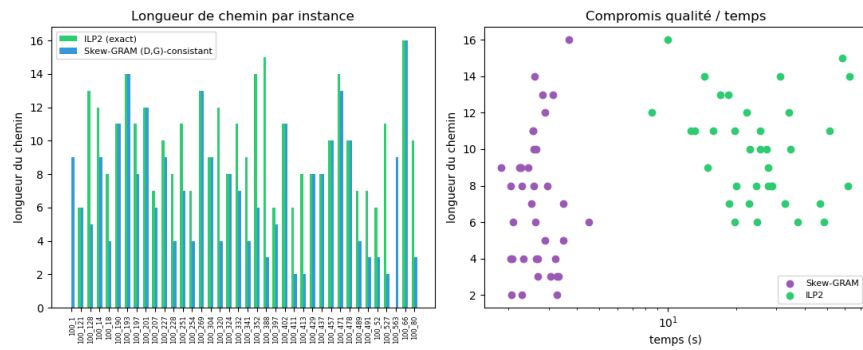
**Table 2.** Comparaison instance par instance, One-to-one Skew-GRAM vs. ILP2, sur 10 instances représentatives de 100 nuds (du meilleur au pire écart).

| Instance | ILP2 (nuds) | Skew-GRAM (nuds) | Écart (%) | Temps ILP2 (s) | Temps Skew-GRAM (s) |
|----------|-------------|------------------|-----------|----------------|---------------------|
| 100_121  | 6           | 6                | 0.0%      | 19.59          | 0.69                |
| 100_402  | 11          | 11               | 0.0%      | 13.16          | 0.80                |
| 100_190  | 11          | 11               | 0.0%      | 25.44          | 0.77                |
| 100_197  | 11          | 9                | -18.2%    | 19.73          | 0.70                |
| 100_14   | 12          | 9                | -25.0%    | 8.51           | 0.77                |
| 100_320  | 12          | 8                | -33.3%    | 33.91          | 0.80                |
| 100_128  | 13          | 8                | -38.5%    | 18.44          | 0.90                |
| 100_66   | 16          | 9                | -43.8%    | 9.98           | 0.97                |
| 100_388  | 15          | 8                | -46.7%    | 58.18          | 0.84                |
| 100_527  | 11          | 4                | -63.6%    | 15.83          | 1.04                |

Le Tableau 2 détaille 10 instances représentatives, allant d’une correspondance exacte avec ILP2 (par ex. 100\_121, 100\_402, 100\_190) jusqu’à l’écart le plus important observé (100\_527) ; sur chacune de ces instances, One-to-one Skew-GRAM reste un à deux ordres de grandeur plus rapide qu’ILP2. Testé sur 38 réseaux d’Erds-Rényi (100 nuds), One-to-one Skew-GRAM extrait des modules fonctionnels de **7.13 nuds** en moyenne, soit un écart moyen de  $-30\%$  par rapport à l’optimum ILP2, et égale ou dépasse ILP2 sur **13 des 36** instances disposant d’un optimum connu. Là où l’état de l’art par Deep Learning échoue (1.2 nuds), la combinaison d’une anticipation exacte sur le DAG, d’embeddings appris et d’un guidage par  $G$  referme une grande partie de l’écart à l’optimum. Le temps d’inférence chute de 28 secondes à **2.7 secondes** en moyenne ( $\sim 10\times$  plus rapide), rendant cette approche viable pour l’analyse de réseaux de plus grande échelle. Retirer l’anticipation DAG (c.-à-d. décoder uniquement par similarité d’embeddings) effondre la longueur moyenne à 5.87 nuds, confirmant qu’éviter les culs-de-sac prématurés dans  $D$  est le facteur dominant des gains



**Figure 4.** Sur une instance réelle de 100 nuds : le chemin brut trouvé dans  $D$  par l'étape A (gauche, rouge) est plus long que le sous-chemin réellement connexe dans  $G$  conservé après l'étape B (droite, bleu) illustrant pourquoi l'étape d'extraction exacte est nécessaire.



**Figure 5.** Longueur de chemin par instance, One-to-one Skew-GRAM vs. ILP2 (gauche), et compromis qualité/temps sur l'ensemble des instances (droite, échelle de temps logarithmique).



observés. Les instances les plus faibles restantes s’expliquent par la variance intrinsèque de l’échantillonnage stochastique à graine fixe, plutôt que par une limite systématique de la fonction de score : sur l’instance la moins bonne, un ré-échantillonnage avec une autre graine retrouve un chemin plus de 4 fois plus long.

## 5 Conclusion

Le problème One-to-one SkewGRAM illustre la complexité d’aligner la causalité et la fonction en biologie. En combinant un embedding de nuds de type Skip-Gram du graphe causal  $D$  avec une anticipation exacte par programmation dynamique et un biais de connexité à  $G$  au moment du décodage, nous avons développé une heuristique environ dix fois plus rapide que le solveur exact ILP2, générant des chemins biologiques nettement plus longs et pertinents que les approches neuronales de bout-en-bout aveugles, tout en restant fidèle à la définition exacte de la (D,G)-consistance grâce à une étape de validation exacte et non incrémentale.

## A Hyperparamètres

Le Tableau 3 donne les valeurs exactes utilisées dans le code pour produire les résultats des Tableaux 1 et 2, avec la graine unique `SEED=42`, pour la reproductibilité.

## Références

1. Ahmed Sidi, M. L., Babou, H. M., Dhib, C., Neron, E., Soukhal, A., Nanne, M. F. : Constraint programming approaches for finding conserved metabolic and genomic patterns. *Computers & Operations Research*, 183 :107166 (2025)
2. Zaharia, A., et al. : CoMetGeNe : Mining conserved neighborhood patterns in metabolic and genomic contexts. *BMC Bioinformatics*, 20(1) :1–20 (2019)
3. Mikolov, T., Sutskever, I., Chen, K., Corrado, G., Dean, J. : Distributed Representations of Words and Phrases and their Compositionality. *NeurIPS* (2013)
4. Perozzi, B., Al-Rfou, R., Skiena, S. : DeepWalk : Online Learning of Social Representations. *KDD* (2014)
5. Grover, A., Leskovec, J. : node2vec : Scalable Feature Learning for Networks. *KDD* (2016)

**Table 3.** Hyperparamètres utilisés par l’Encoder (Phases 1–2) et le Decoder (Phases 3–4).

| Paramètre                         | Symbole    | Valeur | Rôle                                          |
|-----------------------------------|------------|--------|-----------------------------------------------|
| <i>Encoder – Phases 1–2</i>       |            |        |                                               |
| Marches par sommet de départ      | –          | 40     | taille du corpus de marches sur $D$           |
| Longueur maximale d’une marche    | –          | 30     | arrêt anticipé sur un puits de $D$            |
| Fenêtre de contexte               | –          | 3      | paires (centre, contexte)                     |
| Dimension des embeddings          | $d$        | 32     | taille de $\mathbf{V}_{in}, \mathbf{V}_{out}$ |
| Époques d’entraînement            | –          | 8      | optimiseur Adam                               |
| Négatifs structurels par paire    | $K$        | 5      | voir Éq. de la perte, Phase 2                 |
| Taux d’apprentissage              | –          | 0.01   | Adam                                          |
| <i>Decoder – Phases 3–4</i>       |            |        |                                               |
| Marches Monte-Carlo au décodage   | –          | 30 000 | par instance                                  |
| Température du signal d’embedding | $\tau$     | 0.5    | règle de score, Phase 3                       |
| Poids du signal d’embedding       | $\beta$    | 0.3    | règle de score, Phase 3                       |
| Bonus de connexité à $G$          | $\gamma$   | 6.0    | règle de score, Phase 3                       |
| Température de l’anticipation DAG | $\pi_{lf}$ | 1.5    | règle de score, Phase 3                       |